

Chapter 36 — 2D / Grid Dynamic Programming

0. What You Will Learn in This Chapter

2D Dynamic Programming is used when one dimension is not enough to describe the subproblem.

The most common 2D DP patterns are:

- grid DP
- two-index DP
- prefix grid DP
- path counting
- path minimization
- obstacle grids
- triangle grids
- matrix squares
- two-agent grid DP preview
- state with row and column
- state with index and target

This chapter focuses mainly on **grid DP** and general 2D DP foundations.

String-specific 2D DP will be covered in the next chapter.

By the end of this chapter, you should understand:

- What 2D DP means
- How to define `dp[r][c]`
- How to initialize first row and first column
- How to handle obstacles
- How to minimize path sums
- How to count paths
- How to optimize 2D DP space
- How triangle DP differs from rectangle grid DP
- How matrix-square DP works
- How to identify 2D DP from movement constraints
- How to avoid boundary mistakes

Core problems covered in this chapter:

1. Unique Paths
2. Unique Paths II
3. Minimum Path Sum
4. Triangle
5. Maximal Square
6. Count Square Submatrices With All Ones
7. Minimum Falling Path Sum
8. Minimum Falling Path Sum II
9. Dungeon Game
10. Cherry Pickup II, preview
11. Number of Paths With Max Score
12. Out of Boundary Paths
13. Knight Dialer, grid/state preview
14. Paint House, 2D state preview
15. Target Sum, index/target preview

1. What Is 2D Dynamic Programming?

2D DP means the state has two changing variables.

Examples:

```
dp[r][c]
dp[i][j]
dp[index][target]
dp[row][col]
```

In grid DP, the most common meaning is:

```
dp[r][c] = answer for cell (r, c)
```

Example:

Unique Paths:

```
dp[r][c] = number of ways to reach cell (r, c)
```

Minimum Path Sum:

```
dp[r][c] = minimum path sum to reach cell (r, c)
```

Maximal Square:

```
dp[r][c] = side length of largest square ending at cell (r, c)
```

2. Why 2D DP Exists

Some problems need two pieces of information to describe a subproblem.

In a grid, knowing only row is not enough.

You need:

```
row and column
```

Example:

To reach cell (r, c), you may come from:

```
(r - 1, c)
```

```
(r, c - 1)
```

So each cell depends on other cells.

A 2D table naturally stores answers for all cells.

3. Core Need / Why Use 2D DP?

Use 2D DP when:

- input is a grid/matrix
- movement is restricted
- state needs row and column
- state needs index and target

- answer at one cell depends on neighboring cells
 - there are two independent positions or dimensions
-

4. 2D DP vs 1D DP

State	dp[i]	dp[i][j]
Variables	one	two
Common input	array/string/amount	grid/two strings/index+target
Example	House Robber	Unique Paths
Space optimization	often simple	possible but careful

5. When to Use 2D / Grid DP

Use grid DP when problem says:

5.1 Move Through a Grid

Examples:

- Unique Paths
- Minimum Path Sum
- Dungeon Game

5.2 Count Paths

Examples:

- Unique Paths
- Unique Paths II
- Out of Boundary Paths

5.3 Minimize / Maximize Path Cost

Examples:

- Minimum Path Sum
- Minimum Falling Path Sum
- Dungeon Game

5.4 Count or Find Squares

Examples:

- Maximal Square
- Count Square Submatrices

5.5 State Needs Index and Target

Examples:

- Target Sum
 - Knapsack preview
-

6. When NOT to Use 2D DP

6.1 Shortest Path With Arbitrary Obstacles and Four Directions

Use BFS/Dijkstra.

Grid DP works when movement direction is acyclic, such as:

right and down only

or:

from previous row to current row

If you can move up/down/left/right freely, DP order may not exist.

6.2 Need Connected Components

Use DFS/BFS/Union Find.

6.3 Need All Paths Listed

Use backtracking.

6.4 State Needs More Than Two Variables

Use 3D DP, bitmask DP, or state-space BFS.

7. Mental Models

7.1 Fill the Table

Each cell stores answer for a smaller subproblem.

You fill cells in an order where dependencies are ready.

7.2 Movement Reverse Thinking

If you can move right/down forward, then current cell depends on top/left.

7.3 Boundary Rows and Columns

First row and first column often need special initialization.

7.4 Obstacles Block Paths

If a cell is blocked, its DP value is usually 0 or INF.

7.5 Ending at Cell

Many grid DP states mean:

best/count ending at this cell

8. Pattern Recognition Checklist

Ask:

16. Is the input a grid or matrix?
17. Does the problem restrict movement direction?
18. Can I define $dp[r][c]$?
19. Does current cell depend on top/left/diagonal?
20. Are paths counted or costs minimized?
21. Are obstacles present?
22. Is there a base row or column?
23. Can the table be filled row by row?
24. Is the movement acyclic?
25. Can space be optimized to one row?

9. C++ Tools Needed

9.1 2D Vector

```
vector<vector<int>> dp(rows, vector<int>(cols, 0));
```

9.2 INF Initialization

```
const int INF = 1e9;  
vector<vector<int>> dp(rows, vector<int>(cols, INF));
```

9.3 Long Long for Count

```
vector<vector<long long>> dp(rows, vector<long long>(cols, 0));
```

9.4 Direction Arrays

For falling path or neighbor-based DP:

```
vector<int> dirs = {-1, 0, 1};
```

10. Core Templates

Template 1 — Grid Count Paths

```
vector<vector<int>> dp(rows, vector<int>(cols, 0));  
  
dp[0][0] = 1;  
  
for (int r = 0; r < rows; r++) {  
    for (int c = 0; c < cols; c++) {  
        if (r > 0) {  
            dp[r][c] += dp[r - 1][c];  
        }  
  
        if (c > 0) {  
            dp[r][c] += dp[r][c - 1];  
        }  
    }  
}
```

Template 2 — Grid Min Path Sum

```
vector<vector<int>> dp(rows, vector<int>(cols, INF));  
  
dp[0][0] = grid[0][0];  
  
for (int r = 0; r < rows; r++) {
```

```

for (int c = 0; c < cols; c++) {
    if (r > 0) {
        dp[r][c] = min(dp[r][c], dp[r - 1][c] + grid[r][c]);
    }

    if (c > 0) {
        dp[r][c] = min(dp[r][c], dp[r][c - 1] + grid[r][c]);
    }
}
}

```

Template 3 — Obstacle Grid DP

```

if (grid[0][0] == 1) {
    return 0;
}

dp[0][0] = 1;

for (int r = 0; r < rows; r++) {
    for (int c = 0; c < cols; c++) {
        if (grid[r][c] == 1) {
            dp[r][c] = 0;
            continue;
        }

        if (r > 0) {
            dp[r][c] += dp[r - 1][c];
        }

        if (c > 0) {
            dp[r][c] += dp[r][c - 1];
        }
    }
}

```

Template 4 — Space-Optimized Grid DP

```

vector<int> dp(cols, 0);
dp[0] = 1;

for (int r = 0; r < rows; r++) {
    for (int c = 0; c < cols; c++) {

```

```

    if (c > 0) {
        dp[c] += dp[c - 1];
    }
}
}

```

Template 5 — Square DP

```

if (matrix[r][c] == '1') {
    dp[r][c] = 1 + min({
        dp[r - 1][c],
        dp[r][c - 1],
        dp[r - 1][c - 1]
    });
}
}

```

11. Major 2D DP Variations

Count grid paths	dp[r][c] ways	Unique Paths
Obstacle paths	dp[r][c] ways if not blocked	Unique Paths II
Min path cost	dp[r][c] min cost	Minimum Path Sum
Triangle DP	dp[r][c] min from row	Triangle
Square DP	largest square ending at cell	Maximal Square
Falling path	dp[r][c] min falling cost	Falling Path Sum
Reverse grid DP	required future resource	Dungeon Game
Index-target	dp[i][target]	Target Sum / Knapsack
Two-agent grid	dp[row][c1][c2]	Cherry Pickup II

12. Full Problem Walkthrough 1 — LeetCode 62: Unique Paths

12.1 Problem Summary

Robot starts at top-left.

Can move only:

right or down

Return number of ways to reach bottom-right.

12.2 State

$dp[r][c]$ = number of ways to reach cell (r, c)

12.3 Transition

Can come from:

top: $dp[r - 1][c]$

left: $dp[r][c - 1]$

So:

$dp[r][c] = dp[r - 1][c] + dp[r][c - 1]$

12.4 Base Cases

First row:

1 way

First column:

1 way

12.5 C++ Code

```
class Solution {
public:
    int uniquePaths(int m, int n) {
        vector<vector<int>> dp(m, vector<int>(n, 1));

        for (int r = 1; r < m; r++) {
            for (int c = 1; c < n; c++) {
                dp[r][c] = dp[r - 1][c] + dp[r][c - 1];
            }
        }

        return dp[m - 1][n - 1];
    }
};
```

12.6 Complexity

Time:

$O(m * n)$

Space:

$O(m * n)$

13. Full Problem Walkthrough 2 —

LeetCode 63: Unique Paths II

13.1 Problem Summary

Same as Unique Paths, but some cells are obstacles.

1 means obstacle.

Return number of paths.

13.2 State

$dp[r][c]$ = number of ways to reach cell (r, c)

If obstacle:

$dp[r][c] = 0$

13.3 C++ Code

```
class Solution {
public:
    int uniquePathsWithObstacles(vector<vector<int>>& obstacleGrid) {
        int rows = obstacleGrid.size();
        int cols = obstacleGrid[0].size();

        vector<vector<int>> dp(rows, vector<int>(cols, 0));

        if (obstacleGrid[0][0] == 1) {
            return 0;
        }

        dp[0][0] = 1;

        for (int r = 0; r < rows; r++) {
            for (int c = 0; c < cols; c++) {
                if (obstacleGrid[r][c] == 1) {
                    dp[r][c] = 0;
                    continue;
                }

                if (r > 0) {
                    dp[r][c] += dp[r - 1][c];
                }

                if (c > 0) {
                    dp[r][c] += dp[r][c - 1];
                }
            }
        }
    }
};
```

```

    }
}

return dp[rows - 1][cols - 1];
}
};

```

13.4 Complexity

Time:

$O(\text{rows} * \text{cols})$

Space:

$O(\text{rows} * \text{cols})$

13.5 Common Mistakes

- Not checking start obstacle
- Incorrect first row/column initialization
- Adding paths through obstacle cells

14. Full Problem Walkthrough 3 — LeetCode 64: Minimum Path Sum

14.1 Problem Summary

Given grid of non-negative numbers.

Start top-left.

Move only right or down.

Return minimum path sum to bottom-right.

14.2 State

$dp[r][c]$ = minimum path sum to reach cell (r, c)

14.3 Transition

$dp[r][c] = \text{grid}[r][c] + \min(dp[r - 1][c], dp[r][c - 1])$

14.4 C++ Code

```

class Solution {
public:
    int minPathSum(vector<vector<int>>& grid) {
        int rows = grid.size();
        int cols = grid[0].size();

        vector<vector<int>> dp(rows, vector<int>(cols, 0));

        dp[0][0] = grid[0][0];
    }
};

```

```

    for (int r = 1; r < rows; r++) {
        dp[r][0] = dp[r - 1][0] + grid[r][0];
    }

    for (int c = 1; c < cols; c++) {
        dp[0][c] = dp[0][c - 1] + grid[0][c];
    }

    for (int r = 1; r < rows; r++) {
        for (int c = 1; c < cols; c++) {
            dp[r][c] = grid[r][c] + min(dp[r - 1][c], dp[r][c - 1]);
        }
    }

    return dp[rows - 1][cols - 1];
}
};

```

14.5 Complexity

Time:

$O(\text{rows} * \text{cols})$

Space:

$O(\text{rows} * \text{cols})$

15. Full Problem Walkthrough 4 — LeetCode 120: Triangle

15.1 Problem Summary

Given triangle array.

Move from row to next row, choosing adjacent numbers.

Return minimum path sum.

15.2 State

Bottom-up:

$dp[c]$ = minimum path sum from current row position c to bottom

15.3 Transition

$dp[c] = \text{triangle}[r][c] + \min(dp[c], dp[c + 1])$

15.4 C++ Code

```

class Solution {
public:

```

```

int minimumTotal(vector<vector<int>>& triangle) {
    vector<int> dp = triangle.back();

    for (int r = triangle.size() - 2; r >= 0; r--) {
        for (int c = 0; c < triangle[r].size(); c++) {
            dp[c] = triangle[r][c] + min(dp[c], dp[c + 1]);
        }
    }

    return dp[0];
}
};

```

15.5 Complexity

Time:

$O(n^2)$

Space:

$O(n)$

16. Full Problem Walkthrough 5 — LeetCode 221: Maximal Square

16.1 Problem Summary

Given binary matrix of '0' and '1', return area of largest square containing only 1s.

16.2 State

$dp[r][c]$ = side length of largest square ending at cell (r, c)

16.3 Transition

If matrix cell is '1':

$dp[r][c] = 1 + \min(\text{top}, \text{left}, \text{top-left})$

because a square can expand only if all three neighbors support it.

16.4 C++ Code

```

class Solution {
public:
    int maximalSquare(vector<vector<char>>& matrix) {
        int rows = matrix.size();
        int cols = matrix[0].size();

        vector<vector<int>> dp(rows + 1, vector<int>(cols + 1, 0));
    }
};

```

```

int bestSide = 0;

for (int r = 1; r <= rows; r++) {
    for (int c = 1; c <= cols; c++) {
        if (matrix[r - 1][c - 1] == '1') {
            dp[r][c] = 1 + min({
                dp[r - 1][c],
                dp[r][c - 1],
                dp[r - 1][c - 1]
            });

            bestSide = max(bestSide, dp[r][c]);
        }
    }
}

return bestSide * bestSide;
}
};

```

16.5 Complexity

Time:

$O(\text{rows} * \text{cols})$

Space:

$O(\text{rows} * \text{cols})$

17. Full Problem Walkthrough 6 — LeetCode 1277: Count Square Submatrices With All Ones

17.1 Problem Summary

Count all square submatrices containing only ones.

17.2 State

Same as Maximal Square:

$dp[r][c]$ = side length of largest all-ones square ending at (r, c)

If largest square ending here has side k , then it contributes:

k

squares:

1x1, 2x2, ..., kxk

17.3 C++ Code

```
class Solution {
public:
    int countSquares(vector<vector<int>>& matrix) {
        int rows = matrix.size();
        int cols = matrix[0].size();

        vector<vector<int>> dp(rows + 1, vector<int>(cols + 1, 0));

        int answer = 0;

        for (int r = 1; r <= rows; r++) {
            for (int c = 1; c <= cols; c++) {
                if (matrix[r - 1][c - 1] == 1) {
                    dp[r][c] = 1 + min({
                        dp[r - 1][c],
                        dp[r][c - 1],
                        dp[r - 1][c - 1]
                    });

                    answer += dp[r][c];
                }
            }
        }

        return answer;
    }
};
```

17.4 Complexity

Time:

$O(\text{rows} * \text{cols})$

Space:

$O(\text{rows} * \text{cols})$

18. Full Problem Walkthrough 7 —

LeetCode 931: Minimum Falling Path Sum

18.1 Problem Summary

Given square matrix.

Start from any cell in first row.

Each step can go to next row:

down
down-left
down-right

Return minimum falling path sum.

18.2 State

$dp[r][c]$ = minimum path sum ending at cell (r, c)

18.3 Transition

```
dp[r][c] = matrix[r][c] + min(  
    dp[r - 1][c],  
    dp[r - 1][c - 1],  
    dp[r - 1][c + 1]  
)
```

18.4 C++ Code

```
class Solution {  
public:  
    int minFallingPathSum(vector<vector<int>>& matrix) {  
        int n = matrix.size();  
  
        vector<vector<int>> dp = matrix;  
  
        for (int r = 1; r < n; r++) {  
            for (int c = 0; c < n; c++) {  
                int best = dp[r - 1][c];  
  
                if (c > 0) {  
                    best = min(best, dp[r - 1][c - 1]);  
                }  
  
                if (c + 1 < n) {  
                    best = min(best, dp[r - 1][c + 1]);  
                }  
            }  
        }  
    }  
};
```

```

        }

        dp[r][c] = matrix[r][c] + best;
    }
}

return *min_element(dp[n - 1].begin(), dp[n - 1].end());
}
};

```

18.5 Complexity

Time:

$O(n^2)$

Space:

$O(n^2)$

Can be optimized to $O(n)$.

19. Full Problem Walkthrough 8 — LeetCode 1289: Minimum Falling Path Sum II

19.1 Problem Summary

Given grid.

Choose one element from each row.

Cannot choose same column in adjacent rows.

Return minimum sum.

19.2 State

$dp[r][c]$ = minimum sum ending at row r , column c

19.3 Naive Transition

$dp[r][c] = grid[r][c] + \min(dp[r - 1][prevCol])$

where $prevCol \neq c$

Naively $O(n^3)$.

Optimize by tracking smallest and second smallest values from previous row.

19.4 C++ Code

```

class Solution {
public:
    int minFallingPathSum(vector<vector<int>>& grid) {
        int n = grid.size();
    }
};

```

```

vector<int> prev = grid[0];

for (int r = 1; r < n; r++) {
    int min1 = -1;
    int min2 = -1;

    for (int c = 0; c < n; c++) {
        if (min1 == -1 || prev[c] < prev[min1]) {
            min2 = min1;
            min1 = c;
        } else if (min2 == -1 || prev[c] < prev[min2]) {
            min2 = c;
        }
    }

    vector<int> curr(n);

    for (int c = 0; c < n; c++) {
        int bestPrev = (c != min1) ? prev[min1] : prev[min2];
        curr[c] = grid[r][c] + bestPrev;
    }

    prev = curr;
}

return *min_element(prev.begin(), prev.end());
};

```

19.5 Complexity

Time:

$O(n^2)$

Space:

$O(n)$

20. Full Problem Walkthrough 9 — LeetCode 174: Dungeon Game

20.1 Problem Summary

Knight starts at top-left and must rescue princess at bottom-right.
Grid cells can add or subtract health.

Knight dies if health drops to 0 or below.

Return minimum initial health needed.

20.2 Why Forward DP Is Hard

At a cell, knowing max health remaining is not enough.

Need minimum health required to survive from current cell to end.

So solve backwards.

20.3 State

$dp[r][c]$ = minimum health needed before entering cell (r, c)

20.4 Transition

Need enough health after current cell to survive next step.

$need = \min(dp[r+1][c], dp[r][c+1]) - \text{dungeon}[r][c]$

$dp[r][c] = \max(1, need)$

20.5 C++ Code

```
class Solution {
public:
    int calculateMinimumHP(vector<vector<int>>& dungeon) {
        int rows = dungeon.size();
        int cols = dungeon[0].size();

        const int INF = 1e9;

        vector<vector<int>> dp(rows + 1, vector<int>(cols + 1, INF));

        dp[rows][cols - 1] = 1;
        dp[rows - 1][cols] = 1;

        for (int r = rows - 1; r >= 0; r--) {
            for (int c = cols - 1; c >= 0; c--) {
                int need = min(dp[r + 1][c], dp[r][c + 1]) - dungeon[r][c];

                dp[r][c] = max(1, need);
            }
        }

        return dp[0][0];
    }
};
```

20.6 Complexity

Time:

$O(\text{rows} * \text{cols})$

Space:

$O(\text{rows} * \text{cols})$

20.7 Key Lesson

Some grid DP must be solved from destination backwards.

21. Full Problem Walkthrough 10 — LeetCode 1463: Cherry Pickup II, Preview

21.1 Problem Summary

Two robots start at top row:

robot1 at column 0

robot2 at column cols - 1

Both move down one row at a time.

Each can move:

col - 1, col, col + 1

Collect maximum cherries.

21.2 Why This Is 3D DP

State needs:

row, robot1 column, robot2 column

So:

$dp[\text{row}][c1][c2]$

This is beyond basic 2D DP, but it grows from grid DP.

21.3 State

$dp[r][c1][c2]$ = max cherries from row r onward

21.4 Lesson

When one position is not enough, add dimensions.

This problem will be revisited in advanced DP.

22. Full Problem Walkthrough 11 — LeetCode 1301: Number of Paths With Max Score

22.1 Problem Summary

Board contains digits, obstacles X, start S, end E.

Move from bottom-right to top-left using:

up
left
diagonal up-left

Return:

[maxScore, numberOfPaths]

22.2 State

At each cell store:

best score
number of ways to get best score

This is 2D DP with pair state.

22.3 Key Lesson

Sometimes $dp[r][c]$ is not a single number.

It can be:

pair<score, count>

or a struct.

23. Full Problem Walkthrough 12 — LeetCode 576: Out of Boundary Paths

23.1 Problem Summary

Given grid, starting position, and max moves.
Count paths that move out of boundary.

23.2 State

Need position and moves remaining:

$dp[moves][r][c]$

This is technically 3D DP.
But it is grid-state DP.

23.3 Recurrence

From (r,c) with moves left:

sum of paths from four neighbors with moves - 1

If move goes outside, count 1.

23.4 Lesson

When time/move count matters, add it to state.

24. Full Problem Walkthrough 13 — Paint House

24.1 Problem Summary

There are houses and 3 colors.
Adjacent houses cannot have same color.
Cost matrix:

```
costs[i][color]
```

Return minimum total cost.

24.2 State

```
dp[i][color] = min cost to paint houses 0..i with house i painted color
```

24.3 Transition

```
dp[i][0] = costs[i][0] + min(dp[i - 1][1], dp[i - 1][2])
dp[i][1] = costs[i][1] + min(dp[i - 1][0], dp[i - 1][2])
dp[i][2] = costs[i][2] + min(dp[i - 1][0], dp[i - 1][1])
```

24.4 C++ Code

```
int minCost(vector<vector<int>>& costs) {
    int n = costs.size();

    vector<vector<int>> dp = costs;

    for (int i = 1; i < n; i++) {
        dp[i][0] += min(dp[i - 1][1], dp[i - 1][2]);
        dp[i][1] += min(dp[i - 1][0], dp[i - 1][2]);
        dp[i][2] += min(dp[i - 1][0], dp[i - 1][1]);
    }

    return min({dp[n - 1][0], dp[n - 1][1], dp[n - 1][2]});
}
```

24.5 Lesson

2D DP does not always mean a literal grid.
Sometimes dimensions are:

```
index and color/state
```

25. Full Problem Walkthrough 14 — Target Sum, Index/Target Preview

25.1 Problem Summary

Given nums and target.

Assign + or - before every number so expression equals target.

Return number of ways.

25.2 State

A natural DP is:

`dp[index][sum]` = number of ways after processing index with current sum

Because sum can be negative, often use offset or hash map.

25.3 Lesson

If state has:

index + running sum

that is 2D DP conceptually.

This will be covered more deeply in knapsack/target DP.

26. Mini Problem 1 — Unique Paths Space Optimized

```
class Solution {
public:
    int uniquePaths(int m, int n) {
        vector<int> dp(n, 1);

        for (int r = 1; r < m; r++) {
            for (int c = 1; c < n; c++) {
                dp[c] += dp[c - 1];
            }
        }

        return dp[n - 1];
    }
};
```

State:

`dp[c]` = ways to reach current row, column c

27. Mini Problem 2 — Minimum Path Sum Space Optimized

```

class Solution {
public:
    int minPathSum(vector<vector<int>>& grid) {
        int rows = grid.size();
        int cols = grid[0].size();

        vector<int> dp(cols, 0);

        dp[0] = grid[0][0];

        for (int c = 1; c < cols; c++) {
            dp[c] = dp[c - 1] + grid[0][c];
        }

        for (int r = 1; r < rows; r++) {
            dp[0] += grid[r][0];

            for (int c = 1; c < cols; c++) {
                dp[c] = grid[r][c] + min(dp[c], dp[c - 1]);
            }
        }

        return dp[cols - 1];
    }
};

```

28. Mini Problem 3 — Minimum Falling Path Sum Space Optimized

```

class Solution {
public:
    int minFallingPathSum(vector<vector<int>>& matrix) {
        int n = matrix.size();

        vector<int> prev = matrix[0];

        for (int r = 1; r < n; r++) {
            vector<int> curr(n);

            for (int c = 0; c < n; c++) {

```

```

        int best = prev[c];

        if (c > 0) {
            best = min(best, prev[c - 1]);
        }

        if (c + 1 < n) {
            best = min(best, prev[c + 1]);
        }

        curr[c] = matrix[r][c] + best;
    }

    prev = curr;
}

return *min_element(prev.begin(), prev.end());
}
};

```

29. Mini Problem 4 — Maximal Square Space Optimized

```

class Solution {
public:
    int maximalSquare(vector<vector<char>>& matrix) {
        int rows = matrix.size();
        int cols = matrix[0].size();

        vector<int> dp(cols + 1, 0);
        int best = 0;
        int prevDiagonal = 0;

        for (int r = 1; r <= rows; r++) {
            prevDiagonal = 0;

            for (int c = 1; c <= cols; c++) {
                int temp = dp[c];

                if (matrix[r - 1][c - 1] == '1') {
                    dp[c] = 1 + min({dp[c], dp[c - 1], prevDiagonal});
                }
            }
        }
    }
};

```

```
        best = max(best, dp[c]);
    } else {
        dp[c] = 0;
    }

    prevDiagonal = temp;
}

return best * best;
};
```

30. Common 2D DP Edge Cases

30.1 One Row

Only horizontal transitions.

30.2 One Column

Only vertical transitions.

30.3 Start Blocked

Unique Paths II should return 0.

30.4 End Blocked

Answer may be 0.

30.5 Negative Values

Dungeon Game and Falling Path Sum may contain negatives.

30.6 Large Counts

Use modulo or long long if needed.

30.7 Boundary Access

Avoid accessing:

```
dp[-1][c]
dp[r][-1]
```

Use extra padded row/column or boundary checks.

31. Common 2D DP Mistakes

Mistake 1 — Wrong Fill Order

If $dp[r][c]$ depends on top and left, fill top-to-bottom and left-to-right.

Mistake 2 — Bad First Row / Column Initialization

Many grid problems fail due to incorrect boundaries.

Mistake 3 — Treating Obstacles as Normal Cells

Blocked cells must reset ways to 0.

Mistake 4 — Using DP When Movement Has Cycles

If you can move in all four directions, BFS/Dijkstra may be needed.

Mistake 5 — Space Optimizing Too Early

First write clear 2D DP, then optimize.

32. Interview Explanation Templates

32.1 Unique Paths

I define $dp[r][c]$ as the number of ways to reach cell (r,c) . Since movement is only right or down, each cell can only be reached from top or left.

32.2 Minimum Path Sum

I define $dp[r][c]$ as the minimum sum needed to reach (r,c) . The transition takes the minimum of top and left plus current grid value.

32.3 Obstacle Grid

Obstacle cells contribute zero paths. For non-obstacle cells, paths come from top and left.

32.4 Maximal Square

For a cell to be the bottom-right corner of a square, its top, left, and top-left neighbors must also support smaller squares. So the side length is one plus the minimum of those three.

32.5 Dungeon Game

Forward health is hard to compare, so I solve backward. $dp[r][c]$ means minimum health needed before entering this cell to survive to the end.

33. Complexity Patterns

Unique Paths	$O(mn)$	$O(mn)$, $O(n)$ optimized
Unique Paths II	$O(mn)$	$O(mn)$, $O(n)$ optimized
Minimum Path Sum	$O(mn)$	$O(mn)$, $O(n)$ optimized
Triangle	$O(n^2)$	$O(n)$
Maximal Square	$O(mn)$	$O(mn)$, $O(n)$ optimized
Count Squares	$O(mn)$	$O(mn)$, $O(n)$ optimized
Falling Path Sum	$O(n^2)$	$O(n^2)$, $O(n)$ optimized
Falling Path Sum II	$O(n^2)$ optimized	$O(n)$
Dungeon Game	$O(mn)$	$O(mn)$, $O(n)$ optimized

Problem Type	Time	Space
Paint House	$O(n * \text{colors})$	$O(n * \text{colors})$, $O(\text{colors})$ possible
Out of Boundary Paths	$O(mn * \text{moves})$	$O(mn)$ optimized

34. Time-Boxed Practice Guidance

Easy 2D DP

Target:

20–30 minutes

Problems:

- Unique Paths
- Minimum Path Sum
- Triangle

Medium 2D DP

Target:

30–50 minutes

Problems:

- Unique Paths II
- Maximal Square
- Count Square Submatrices
- Minimum Falling Path Sum
- Paint House

Hard 2D DP

Target:

50–90 minutes

Problems:

- Dungeon Game
- Minimum Falling Path Sum II
- Out of Boundary Paths
- Cherry Pickup II
- Number of Paths With Max Score

35. Expanded Practice Set

35.1 Must-Do Medium

26. Unique Paths
27. Unique Paths II
28. Minimum Path Sum
29. Triangle
30. Maximal Square
31. Count Square Submatrices With All Ones
32. Minimum Falling Path Sum

- 33. Paint House
- 34. Dungeon Game

35.2 Strong Interview Practice

- 35. Minimum Falling Path Sum II
- 36. Number of Paths With Max Score
- 37. Out of Boundary Paths
- 38. Cherry Pickup II
- 39. Knight Dialer
- 40. Target Sum
- 41. Minimum Path Cost in a Grid
- 42. Paths in Matrix Whose Sum Is Divisible by K

35.3 Hard / Stretch

- 43. Cherry Pickup
 - 44. Cherry Pickup II
 - 45. Dungeon Game
 - 46. Number of Paths With Max Score
 - 47. Minimum Falling Path Sum II
 - 48. Count All Possible Routes
 - 49. Profitable Schemes
 - 50. Knight Probability in Chessboard
-

36. Suggested Study Order

Step 1 — Basic Grid Count

Start with:

- 51. Unique Paths
- 52. Unique Paths II

Goal:

Learn path counting and obstacles.

Step 2 — Grid Min Cost

Solve:

- 53. Minimum Path Sum
- 54. Triangle
- 55. Minimum Falling Path Sum

Goal:

Learn minimizing path cost.

Step 3 — Square DP

Solve:

- 56. Maximal Square
- 57. Count Square Submatrices

Goal:

Learn top/left/diagonal dependency.

Step 4 — Reverse DP

Solve:

- 58. Dungeon Game

Goal:

Learn solving backward from destination.

Step 5 — Extra State Grid DP

Solve:

- 59. Out of Boundary Paths
- 60. Cherry Pickup II
- 61. Knight Dialer

Goal:

Learn adding dimensions for moves/agents/state.

37. Pattern Decision Table

robot moves right/down	$dp[r][c]$ ways	Unique Paths
obstacles block cells	$dp[r][c] = 0$ if blocked	Unique Paths II
min sum path	$dp[r][c]$ min cost	Minimum Path Sum
triangle adjacent moves	row-based DP	Triangle
largest all-ones square	square ending at cell	Maximal Square
count all squares	add side length	Count Squares
falling path	previous row transitions	Falling Path Sum
cannot use same column	min and second min	Falling Path Sum II
survive from start to end	reverse DP	Dungeon Game
adjacent house colors	index + color	Paint House
move count matters	moves + row + col	Out of Boundary Paths
two robots	row + col1 + col2	Cherry Pickup II

38. Mini Problem 5 — Paths in Matrix Whose Sum Is Divisible by K, Preview

State:

$dp[r][c][mod]$ = number of paths to cell (r,c) with sum mod k

Transition from top and left.

This is grid DP with an extra modulo dimension.

39. Mini Problem 6 — Knight Probability in Chessboard, Preview

State:

$dp[move][r][c]$ = probability knight is at cell (r,c) after move

Each move distributes probability to 8 directions.

This is 3D DP over moves and board position.

40. Common Edge Case Checklist

Before submitting 2D DP, test:

1x1 grid
one row
one column
start obstacle
end obstacle
all obstacles
all zeros
negative values
large values
empty triangle
single-row triangle
matrix with only one 1
matrix with no 1

41. Debugging 2D DP

If your grid DP fails:

62. Define $dp[r][c]$ aloud.
63. Check what cells can lead into (r,c) .
64. Check first row and first column.
65. Check obstacle handling.
66. Check fill order.
67. Check final answer cell.
68. Check if movement has cycles.
69. Check if space optimization overwrites needed values.

42. What to Master Before Chapter 37

Before moving to String Dynamic Programming, make sure you can:

- Define `dp[r][c]`
 - Initialize first row and column
 - Solve Unique Paths
 - Solve Unique Paths II
 - Solve Minimum Path Sum
 - Solve Triangle
 - Solve Maximal Square
 - Solve Count Square Submatrices
 - Solve Minimum Falling Path Sum
 - Understand reverse DP from destination
 - Understand when grid DP fails and BFS is needed
 - Space optimize row-based grid DP
 - Recognize when extra state dimension is required
-

43. Final Cheat Sheet

Count Paths

```
dp[r][c] = dp[r - 1][c] + dp[r][c - 1];
```

Min Path Sum

```
dp[r][c] = grid[r][c] + min(dp[r - 1][c], dp[r][c - 1]);
```

Obstacle Cell

```
if (obstacleGrid[r][c] == 1) {
    dp[r][c] = 0;
}
```

Maximal Square

```
if (matrix[r][c] == '1') {
    dp[r][c] = 1 + min({top, left, diagonal});
}
```

Falling Path

```
dp[r][c] = matrix[r][c] + min(previous row allowed columns);
```

Dungeon Game

```
dp[r][c] = max(1, min(dp[r + 1][c], dp[r][c + 1]) - dungeon[r][c]);
```

44. Core Takeaway

2D DP is used when one index is not enough.

For grid problems, the main question is:

What does `dp[r][c]` represent?

Then ask:

Which previous cells can reach this cell?

Most grid DP depends on:

- top
- left
- diagonal
- previous row

The most common mistakes are boundary initialization and using DP when movement creates cycles.

If movement is acyclic, grid DP is often perfect.

If movement can go freely in all directions, consider BFS or Dijkstra instead.
